Lang Min, Shuhan Dong
Prof. John Guerra
CS 7180
22 March 2026

# Homework 4 Reflection

## 1. Explore → Plan → Implement → Commit vs. Previous Approach

In our PreLeave project (P2), development was ad-hoc: scan a few files, start coding, and fix structural problems mid-flight. Commits mixed features, bug fixes, and refactors with no clear narrative. The four-phase pattern changed this fundamentally.

**Explore** Claude Code scanned all configs, the empty Prisma schema, and 6 zero-byte route files, producing a structured 10-item gap analysis for US-1 (Treatment Logging). This replaced guesswork with a systematic inventory.

**Plan** Plan mode generated a 10-file implementation sequence with dependency ordering—shared infrastructure before feature code—eliminating the rework cycles we experienced in P2.

**Implement** 16 files were generated in under 6 minutes, all consistent with CLAUDE.md conventions: asyncHandler on every route, Zod validation on every endpoint, no PHI in log output.

**Commit** A review phase caught 3 bugs (app.listen in tests, void-return middleware, logger type issue) before they reached the repository—bugs that would have become separate fix commits days later.

The result: Treatment Logging has 3 clean commits; Lab Results (TDD) has 3 more (red → green → refactor). Each commit tells a clear story—a stark contrast to P2's tangled history.

## 2. Context Management Strategies

**/clear** Best for switching concerns. After implementing Treatment Logging (16 files), we cleared context before starting TDD on Lab Results to prevent implementation bias in test design.

**/compact** Ideal after debugging. During the Green phase we hit a bcrypt native binary error and a JWT mock incompatibility, generating long stack traces. Compacting preserved the fix while shedding the noise.

**--continue** Bridged sessions effectively. After committing Treatment Logging, we resumed the same context for Lab Results TDD, preserving project awareness without re-exploring.

**Security incident as a lesson:** On March 21, I accidentally executed a malicious command from a phishing site impersonating Claude Code's download page. The malware gained root access, installed a persistent backdoor (LaunchDaemon + KeepAlive), and ran C2 communication for ~9 minutes before I detected and removed it. This made our CLAUDE.md deny-rules (`Deny: Bash(curl *)`, `Deny: Bash(sudo *)`) feel critical rather than academic—the attack exploited exactly the kind of command our permission configuration blocks.

## 3. Annotated Claude Code Session Log

**Explore — Gap Analysis**

```
❯ Explore the current project structure. Use Glob to find all config files...
  What We Have: Config (complete), Source Files (all empty, 0 bytes)
  What's Missing for US-1: Prisma models, Express bootstrap, Auth, CRUD routes...
```

→ *In P2 we would have opened random files until we found what we needed. Here, Claude Code read 4 config files and checked all 6 route files in seconds, producing a structured gap table. The 10-item action list gave us a concrete implementation roadmap before writing a single line of code. Key insight: the Explore phase revealed that ALL source files were 0 bytes and Prisma had zero models—without it we might have assumed some scaffolding existed.*

**Plan — Sequenced Implementation**

```
❯ [Plan mode] Plan the implementation of Treatment Logging (US-1)...
  10-file plan: schema → constants → logger → client → middleware → service →
routes
```

→ *Plan mode forced us to think about dependency ordering before coding. The plan put shared infrastructure (logger, Prisma client, asyncHandler) at steps 2–5 and feature code at steps 7–10. In P2, we wrote routes first and retrofitted middleware later, causing circular import issues. The plan also surfaced design decisions early—making uricAcidLevel optional came from reading PRD acceptance criteria ("not every session includes a lab draw"), which we would have missed if we had jumped straight to coding.*

**Implement — Execute the Plan**

```
❯ Implement the plan. Start with Prisma schema, then Zod validators, service,
routes...
  Created 16 files: schema, constants, logger, client, asyncHandler, auth,
validators,
  types, services, routes, Express app, and tests – completed in 5m 27s
```

→ *The most striking result: 16 files generated in 5m 27s, and every one followed CLAUDE.md conventions without us having to remind Claude Code. asyncHandler wrapped every async route, Zod validated every*

*endpoint, the logger stripped PHI fields per ADR-6, and JWT tokens went into httpOnly cookies per ADR-3. This proved that a well-written CLAUDE.md is not just documentation—it is the AI's operating manual that directly controls code quality.*

**Commit — Review & Clean Commits**

```
❯ Review the implementation. Check asyncHandler, Zod validation, no PHI logged...
  Found 3 bugs: app.listen in tests, void-return auth middleware, logger type
spread
  git commit -m "fix: guard app.listen, fix auth middleware, tighten logger"
```

*→ The review step was the biggest surprise. Claude Code re-read all 7 implementation files and found 3 bugs we would have shipped: (1) app.listen() running during test imports causing port 3001 conflicts, (2) return next() in void-typed middleware that TypeScript strict mode would flag, (3) implicit undefined spread in the logger format. In P2 these would have surfaced as separate bug-fix commits days later. The Commit phase turned them into a single clean fix before the code even reached the repo.*

**TDD Red — Tests Before Code**

```
❯ Write failing tests FIRST for Lab Result feature...
  FAIL: Cannot find module '../../services/labService'
```

*→ Writing tests first forced us to define the API contract before building it. The "Cannot find module" errors were exactly what we wanted—tests failing because the implementation does not exist yet, not because the tests themselves are broken. This distinction matters: a test that fails due to a typo is noise, but a test that fails due to missing code is a specification. The 3 files covered validation boundaries (uric acid > 0 and ≤ 20), service logic (date sorting, user scoping), and integration (auth + validation + response shape).*

**TDD Green — Minimum to Pass**

```
❯ Implement MINIMUM code to make all lab result tests pass...
  Test Files  3 passed (3)  |  Tests  40 passed (40)
```

*→ Claude Code excelled here—given failing tests, it produced the minimum code to make them pass and nothing more. No premature optimization, no extra endpoints, no features the tests did not ask for. The JWT mock fix was a real debugging challenge: Vitest's ESM interop required returning { ...mocked, default: mocked } because auth.ts uses import jwt from 'jsonwebtoken' which resolves to the default export. Without the explicit default override, every authenticated route returned 401. This is the kind of environment-specific issue that TDD surfaces early.*

**TDD Refactor — Extract & Clean**

```
❯ Refactor: extract shared validation logic...
  Created shared.ts (isoDatetimeField, notesField, dateRangeQuerySchema)
  Test Files  6 passed (6)  |  Tests  81 passed (81)
```

*→ The refactor demonstrated why the green phase should be minimal. With 81 passing tests as a safety net, we could confidently extract shared.ts (3 reusable Zod fields used by both validators), improve error*

*messages to include units (mg/dL), and add JSDoc without fear of breaking anything. The key lesson: refactoring is where human judgment matters most. Claude Code can extract code mechanically, but deciding what to name the shared module and which abstractions to create requires understanding the domain. If we had to redo this, we would also extract the JWT mock fix into a shared test helper rather than duplicating it in two integration test files.*